Criando Objetos com Python

Mário Leite

Python é uma linguagem moderna, interpretada e de sintaxe extremamente clara, criada por *Guido van Rossum*, ela prioriza legibilidade e produtividade; é usada em automação, ciência de dados, web, IA, ensino e lógica. Seu modelo orientado a objetos é simples e poderoso; não exige declarações de tipo, o reduz a "cerimônia" do código; isto permite focar no conceito e não na burocracia sintática, facilitando o ensino de algoritmos de maneira bem didática. E para aplicações reais, é extremamente produtiva; a única reclamação de muitos programadores iniciantes é a tal "indentação" obrigatória. Embora alguns "pythonistas" acham isto importante para organizar o código, os iniciantes reclamam: pois se apenas um espaço estiver fora do bloco, em um método de um programa de 300 linhas, tudo estará perdido. Por outro lado, a despeito desse pequeno inconveniente, o Python é, realmente, muito poderoso. Esta postagem mostra um dos recursos mais importantes da linguagem: a criação de objetos de maneira simples.

Conceito de Objeto em Python

Em Python:

- Classe → define o modelo
- Objeto → é uma instância da classe

A criação é direta e sem palavras extras.

Exemplo 1 - Classe básica

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def Apresentar(self):
        print(f"Olá, eu sou {self.nome} e tenho {self.idade} anos.")
```

✓ class Pessoa → definição da classe

✓ `__init__` → construtor

✓ **self** → referência ao próprio objeto

✓ atributos criados dinamicamente

Criando os objetos p1, p2 (instâncias da classe "Pessoa")

```
p1 = Pessoa("Ana", 30)
p2 = Pessoa("Carlos", 25)

p1.Apresentar()
p2.Apresentar()
```

Saída

Olá, eu sou Ana e tenho 30 anos.
Olá, eu sou Carlos e tenho 25 anos.

Observe: não há **new**, não há tipo obrigatório, não há *boilerplate*.

Exemplo 2 - Objeto com regra de negócio

```
class Aluno:
    def __init__(self, nome, n1, n2):
        self.nome = nome
        self.n1 = n1
        self.n2 = n2

    def media(self):
        return (self.n1 + self.n2) / 2

    def situacao(self):
        m = self.media()
        if m >= 7:
            return "APROVADO"
        elif m >= 5:
            return "RECUPERAÇÃO"
        else:
            return "REPROVADO"
```

Uso do objeto

```
aluno = Aluno("João", 6.5, 8.0)
print("Aluno :", aluno.nome)
print("Média :", aluno.media())
print("Situação:", aluno.situacao())
```

Saída

Aluno : João
Média : 7.25
Situação: APROVADO

Características importantes do modelo OO em Python:

✓ Criação dinâmica de atributos
Os atributos nascem no `__init__`, sem declaração prévia.
✓ Métodos são funções da classe
Sempre recebem self.
✓ Tudo é objeto
Números, strings, listas — tudo.
✓ Simplicidade sintática
Menos código, mais foco no comportamento.

Comparação conceitual (didática)

Conceito	Python
Construtor	`__init__`
Objeto	**obj** = **Classe**()
Atributos	Criados dinamicamente
Encapsulamento	Convencional (_atributo)
Tipagem	Dinâmica

Conclusão: A criação de objetos em Python é:
✓ Direta
✓ Legível
✓ Sem burocracia sintática
✓ Ideal para ensino de orientação a objetos
Permite focar no modelo do problema e não na linguagem.